

Week-8 (DAY-5) – Handson – Keerthivasan R



Problem Statement

In modern software development, applications must be reliable, maintainable, and error-free. However, many developers deploy applications without proper testing, which leads to:

- Unexpected runtime errors
- Business logic failures
- Incorrect calculations
- Security vulnerabilities
- Difficult debugging and maintenance

To ensure software quality, automated unit testing must be implemented.

This project aims to implement **unit testing using NUnit framework** in a .NET / ASP.NET Core application to validate business logic and ensure the correctness of application components.



Objective

Develop and test a sample ASP.NET Core application using **NUnit** to:

- Verify business logic functionality
 - Test service layer methods
 - Validate input and output conditions
 - Handle exception testing
 - Improve code reliability
-



System Description

Consider a simple **Banking / Insurance / Employee Management System** with the following operations:

- Add Employee / Customer
- Calculate Salary / Premium
- Process Claims
- Perform Deposit / Withdrawal
- Generate Reports

The goal is to write **NUnit test cases** for:

1. Valid input scenarios
 2. Invalid input scenarios
 3. Boundary value testing
 4. Exception handling
 5. Business rule validation
-



Technical Requirements

- .NET 6 / .NET 7 / .NET 8
 - NUnit Framework
 - NUnit3TestAdapter
 - Microsoft.NET.Test.Sdk
 - Visual Studio
-



Example Testing Scenarios

- Test if salary calculation returns correct value
 - Test if withdrawal fails when balance is insufficient
 - Test if claim amount cannot be negative
 - Test if exception is thrown for null inputs
 - Test API response status codes
-



Expected Outcome

- All business logic is validated through automated test cases
 - Bugs are identified early in development
 - Code becomes maintainable and refactor-friendly
 - Application reliability increases
-

OUTPUT:

EmployeeManagement.API 1.0 OAS 3.0

<https://localhost:7251/EmployeeManagement.API>

WeatherForecast

GET /weatherForecast

Parameters

No parameters

Execute Clear

Responses

Curl

```
curl -X GET \
  'https://localhost:7251/WeatherForecast' \
  -H 'accept: text/plain'
```

Request URL

```
https://localhost:7251/WeatherForecast
```

Server response

Code

Details

200

Response body

```
{
  "date": "2025-04-20",
  "temperature": 10,
  "temperature": 10,
  "summary": "Chilly"
},
{
  "date": "2025-04-21",
  "temperature": 10,
  "temperature": 10,
  "summary": "Bracing"
},
{
  "date": "2025-04-22",
  "temperature": 10,
  "temperature": 10,
  "summary": "Cool"
},
{
  "date": "2025-04-23",
  "temperature": 10,
  "temperature": 10,
  "summary": "Freezing"
},
{
  "date": "2025-04-24",
  "temperature": 10,
  "temperature": 10,
  "summary": "Freezing"
}
```

Response headers

```
content-type: application/json; charset=utf-8
date: Sun, 19 Apr 2025 14:45:13 GMT
server: Kestrel
```

Responses

Code	Description	Links
200	OK	No links

Media type

text/plain

Content-Accept Header

Example Value : Schema

```
{
  "date": "2025-04-19",
  "temperature": 10,
  "temperature": 10,
  "summary": "string"
}
```

Schemas